



Trabajo Práctico: Concurrency, Channels and System Critique (Go)

Materia: Programación Concurrente

Ciclo Lectivo: 2026

Introducción y Objetivos

El presente trabajo práctico tiene como finalidad profundizar en el modelo de concurrencia de Go (*CSP - Communicating Sequential Processes*). Se evaluará no solo la capacidad de escribir código que funcione, sino la habilidad para detectar bloqueos (*deadlocks*), gestionar el ciclo de vida de las goroutines y mantener una postura crítica frente a soluciones automatizadas (IA).

Parte I: El Acertijo de los Canales (Comprensión)

Analicen el siguiente fragmento de código que intenta enviar mensajes de log a un sistema de monitoreo.

```
package main
import "fmt"

func main() {
    ch := make(chan string, 2)
    ch <- "Log 1: Sistema iniciado"
    ch <- "Log 2: Conexión establecida"
    ch <- "Log 3: Error de red detectado" // ¿Qué sucede en esta
    línea?

    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

Consignas:

1. Al ejecutar este código, el programa falla. Expliquen técnicamente **por qué ocurre un "deadlock"** en la tercera inserción al canal.
2. Si eliminamos el número 2 en la creación del canal (`make(chan string)`), el programa

falla mucho antes. Expliquen el concepto de **Rendezvous** (encuentro) y por qué es fundamental para entender los canales sin buffer.

Parte II: El "Bug del Principiante" (Debugging y Sincronización)

Un desarrollador junior entregó el siguiente código para procesar 5 tareas en paralelo. El código compila, pero al ejecutarlo se produce un error crítico de ejecución.

Go

```
func worker(id int, resultados chan string, wg *sync.WaitGroup) {
    defer wg.Done()
    // Simula una tarea pesada
    resultados <- fmt.Sprintf("Resultado de tarea %d", id)
}

func main() {
    resultados := make(chan string)
    var wg sync.WaitGroup

    for i := 1; i <= 5; i++ {
        wg.Add(1)
        go worker(i, resultados, &wg)
    }

    wg.Wait()      // Línea de conflicto A
    close(resultados)

    for res := range resultados {
        fmt.Println(res)
    }
}
```

Consignas:

1. Identifiquen el **Deadlock Circular**: Expliquen por qué el `main` queda bloqueado en `wg.Wait()` mientras los `workers` quedan bloqueados intentando enviar datos al canal `resultados`.
 2. Propongan una solución técnica (por ejemplo, el uso de una goroutine adicional para el cierre del canal o el uso de buffers) y **justifiquen por qué su solución elimina el bloqueo**.
-

Parte III: Generación del Código Base con IA

Utilizar IA de su elección con el siguiente prompt exacto y adjuntar el código generado en su entrega:

"Actúa como un programador Junior en Go. Diseña un sistema de alta disponibilidad para la aprobación y análisis de fraude de transacciones financieras concurrentes. Una función debe cargar un lote secuencial de exactamente 50 transacciones enviándolas a un canal sin buffer llamado `transacciones_entrantes`. Instanciá exactamente 5 goroutines de validación (workers) en paralelo que consuman de ahí, tardando de forma aleatoria entre 100ms y 400ms por operación. Los workers deben enviar el string 'Transacción X aprobada por el Nodo Y' a un canal llamado `registro_contable`. El main debe imprimir los resultados y finalizar de forma limpia usando `sync.WaitGroup` y `close()` sin usar bloqueos mágicos de tiempo como `time.Sleep`."

Parte IV: Informe de Auditoría y Resiliencia

Una vez obtenido el código, responder:

1. **Análisis de Buffering y Filosofía CSP:** Al ser transacciones financieras en tiempo real, el enunciado exige un canal Unbuffered. Expliquen detalladamente qué propiedad de sincronización matemática (Rendezvous) se perdería si la IA hubiese ignorado el prompt y configurado un canal con buffer.
2. **Trampa del Ciclo de Vida:** Las IA suelen cometer el error de colocar la función `close()` dentro del bucle de los workers o llamar a `close()` sobre un canal mientras otras Goroutines siguen intentando escribir en él. Analicen minuciosamente el código entregado: ¿Sufre el programa del error crítico de lanzar un Panic por escribir en un canal cerrado? Justifiquen analizando la matriz de estados vista en clase.
3. **Refactorización de Resiliencia (Inyección de Select):** Las redes bancarias fallan. Refacturen el código de la IA agregando un `select` con `time.After` dentro del bloque de los workers. Si la consulta simulada tarda más de 250ms, el worker debe abortar la operación, enviar el mensaje "Transacción X rechazada por Timeout de Red" al canal de resultados y continuar con la siguiente transacción.

Entregables

En una carpeta:

1. Código fuente en Go (.go).
2. Informe en PDF con las respuestas de las Partes I, II y IV.
3. Prompts utilizados para la IA en formato PDF, pueden compartir el chat completo o lo que ustedes fueron enviando para llegar al resultado final del TP.